Storage Lab: Block, File & Object Storage Classification & Service Mapping (Local VM)

Duration : 30 Min.

Scenario

A platform architect has drafted an infrastructure plan for a mixed-workload enterprise application consisting of: (1) transactional Oracle database datafiles, (2) shared application server media uploads accessed concurrently by five web nodes, and (3) cold quarterly audit log archives. The junior engineer must correctly classify each workload requirement into Block, File, or Object storage, and map each to the exact native storage services in AWS, Azure, GCP, and on-premises SAN/NAS.

Task Objectives

Perform the following actions inside your `student_workspace` directory:

> **Pre-Staged Files in `student_workspace/`:** - **`storage_mapping.json`**: Starter boilerplate file already provided with the workload entries and `TODO_*` placeholders. - **`workloads_input.json`**: Technical background reference with workload profiles and I/O specifications.

1. Workload Classification

- Analyze the 3 enterprise workload descriptions:
- **Workload 1:** Transactional Oracle database datafiles.
- **Workload 2:** Shared application server media uploads accessed concurrently by five web nodes.
- **Workload 3:** Cold quarterly audit log archives.
- Classify each workload into its respective storage tier: **Block**, **File**, or **Object** storage.

2. Cross-Cloud Service Mapping

- Map each classified workload to its exact native storage services across the major cloud providers:
- **AWS**: EBS, EFS, S3
- **Azure**: Managed Disks, Azure Files, Blob Storage
- **GCP**: Persistent Disk, Filestore, Cloud Storage

3. Artifact Completion

- Directly edit the pre-staged file `storage_mapping.json` inside `student_workspace/` and replace all `TODO_*` placeholder fields with:
- `storage_tier`: The classified tier (`Block`, `File`, or `Object`).

- `protocols`: The associated network and hardware protocols / interfaces (e.g., `iSCSI`/`NVMe` for Block, `NFS`/`SMB` for File, `HTTPS`/`REST` for Object).
- `aws_service`, `azure_service`, `gcp_service`: The corresponding native cloud storage service names.

Verification

Once you have updated `storage_mapping.json`, verify your solution locally by running:

```
python run.py
```

Once verified, click **"Run Tests"** on the assessment platform to record your score.

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	Workload classification: accurate categorization of database, shared media, and audit archives across Block, File, and Object models	4 Marks
TC2	Cross-cloud service mapping: 100% correct mapping to AWS, Azure, and GCP native storage service offerings	3 Marks
TC3	Artifact generation: storage_mapping.json exists, is valid JSON, and contains protocol details (iSCSI/NVMe, NFS/SMB, HTTPS REST)	3 Marks

Total Score: 10 Marks